

Resumen Memoria – Parte practica

Algoritmos de paginación

La técnica de paginación intenta alocar la mayor cantidad de paginas posibles. Cada vez que hay que alocar una pagina en un marco, se produce un **page fault**.

Cuando es necesario alocar una pagina y ya no hay espacios disponibles, se debe seleccionar una **pagina victima** para lo cual existen diversos algoritmos.

Algoritmo FIFO

Trata a los frames en uso como una cola circular.

La pagina mas vieja en memoria es reemplazada.

La ventaja es que es fácil de implementar, la desventaja la pagina puede ser necesitada pronto.

Marco/Pagina	1	2	1	3	4	1	4	3	5
F1 (Frame 1)	1	1	1	1	4	4	4	4	4
F2 (Frame 2)		2	2	2	2	1	1	1	1
F3 (Frame 3)				3	3	3	3	3	4
PF (Page fault)	x	x		x	x	x			x

opcional: para ir guiándose se puede graficar una cola de la siguiente manera:

1 2 3 4 1 5

y lo que hago es ir agregando a medida que se cargan las páginas, para saber cual entro primero y cual saco a la hora de elegir la pagina victima y cuando la elijo la tacho.

Algoritmo FIFO – con segunda chance

Se utiliza un bit adicional llamado **bit de referencia**, cuando la pagina se carga en memoria el bitR (bit de referencia) esta en 0, cuando la pagina es referencia el bitR se pone en 1. La victima se busca en orden FIFO, se selecciona la primera página cuyo bitR está en 0, mientras se busca la victima cada bitR que tiene valor 1 se cambia a 0.

Marco/Pagina	1	2	1	3	4	1	4	3	5
F1 (Frame 1)	1	1	1*	1*	1	1*	1*	1*	1
F2 (Frame 2)		2	2	2	4	4	4*	4*	4
F3 (Frame 3)				3	3	3	3	3*	5
PF (Page fault)	x	x		x	x				x

En esta ocasión es conveniente ir llevando una cola para guiarse de la siguiente manera:

1* 2 3* 4* 1* 3 4 1

Algoritmo LRU

LRU (Last Recently used) (menos recientemente usado) reemplaza la pagina que no fue referenciada por más tiempo.

Cada página debe tener información del instante de su ultima referencia, esto genera un overhead mayor.

	1°	2°	3°	4°	5°	6°	7°	8°	9°
Marco/Pagina	1	2	1	3	4	1	4	3	5
F1 (Frame 1)	1	1	1	1	1	1	1	1	5
F2 (Frame 2)		2	2	2	4	4	4	4	4
F3 (Frame 3)				3	3	3	3	3	3
PF (Page fault)	x	x		x	x				x

En este caso casi que, si o si hay que llevar una cola para no perderse, lo que hago es ver en qué posición entran y elijo la posición mas chica (el numero de arriba es la posición)

1³ 2⁶ 3⁴ 4⁸ 5⁷ 5⁹

Algoritmo Optimo

Selecciona la página cuya próxima referencia se encuentra mas lejana a la actual.

En practica es imposible de implementar ya que no se conoce futuros eventos.

Es como lo opuesto a el LRU

	1°	2°	3°	4°	5°	6°	7°	8°	9°
Marco/Pagina	1	2	1	3	4	1	4	3	5
F1 (Frame 1)	1	1	1	1	1	1	1	1	5
F2 (Frame 2)		2	2	2	4	4	4	4	4
F3 (Frame 3)				3	3	3	3	3	3
PF (Page fault)	x	x		x	x				

En este caso como en LRU debemos llevar la cola y lo que hago es llevar la próxima posición en la que se va a repetir el número, si no tiene próxima posición pongo una x, siempre la x es la **próxima víctima**, sino elijo la **posición mas alta** (como en LRU la posición es el numero de arriba)

1³ 2⁶ 2^x 3⁴ 4⁸ 4^x 5⁷ 5^x

🐧 Paginación – Descarga asincrónica de paginas

El sistema reserva uno o varios marcos para la descarga asincrónica de páginas.

Cuando es necesario descargar una página modificada:

La pagina que provoco el fallo se coloca en un frame designado a la descarga asincrónica.

El SO envía la orden de descargar asincrónicamente la página modificada mientras continua la ejecución de otro proceso.

El frame de descarga asincrónica pasa a ser el que contenía a la página victima que ya se descargó previamente.

• Ejemplo de algoritmo *FIFO*

Marco/ Página	1	2	1 ^M	3	4	3 ^M	5		6	7	
F1	1	1	1 ^M	1 ^M	1 ^M	1 ^M	1 ^M			7	7
F2		2	2	2	2	2	2	2	6	6	6
F3				3	3	3 ^M	3 ^M	3 ^M	3 ^M	3 ^M	
F4					4	4	4	4	4	4	4
F5							5	5	5	5	5

Lo que se hace es que se reserva un marco para la descarga asincrónica de paginas y cuando la pagina victima seleccionada (que puede ser usando cualquiera de los algoritmos mencionados previamente) es una página con el bitM (bit de modificada) lo que se hace es usar ese frame de descarga asincrónica para alocar la pagina que quiere ingresar y donde estaba la página con el bitM pasa a ser el nuevo frame de descarga asincrónica, dejándolo vacío.

🐧 Algoritmos de planificación en un HDD

El objetivo es minimizar el movimiento del cabezal ordenando lógicamente los requerimientos pendientes (que están en la cola) al disco, considerando el numero de cilindro de cada requerimiento. En cualquier momento se pueden encolar nuevos movimientos.

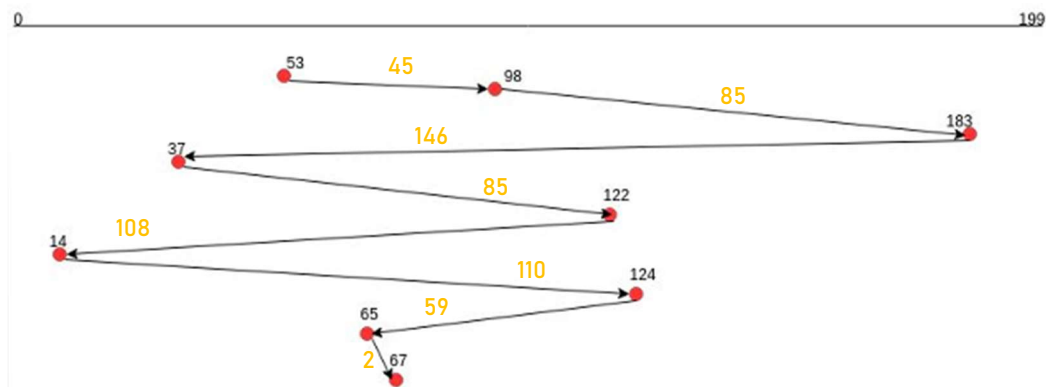
La atención de requerimientos a pistas duplicadas se resuelven según el algoritmo de planificación.

Algoritmos de planificación: Ejemplo sin page faults:

FCFS – First Come First Served

Atiende los requerimientos por orden de llegada

- Cantidad de pistas: 200 (0..199)
- Requerimientos en la cola: { 98, 183, 37, 122, 14, 124, 65, 67 }
- Viene de: pista 61
- Ubicación actual de cabezal: pista 53 -> derecha-izquierda

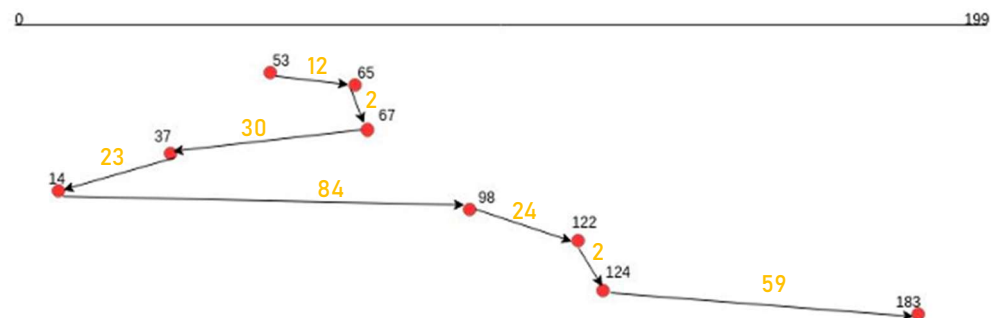


Movimientos: 640

SSTF – Shortest Seek Time First

Selecciona el requerimiento que requiere le menor movimiento del cabezal.

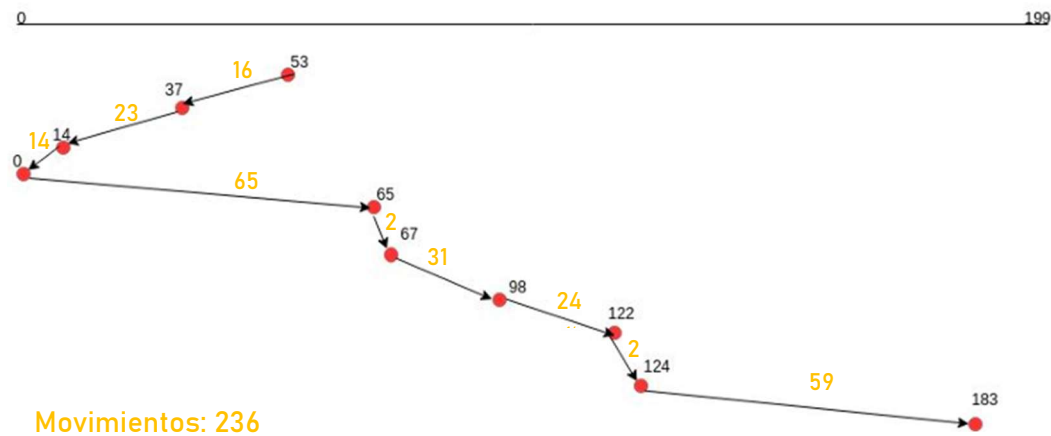
- Cantidad de pistas: 200 (0..199)
- Requerimientos en la cola: { 98, 183, 37, 122, 14, 124, 65, 67 }
- Viene de: pista 61
- Ubicación actual de cabezal: pista 53 -> derecha-izquierda



Movimientos: 236

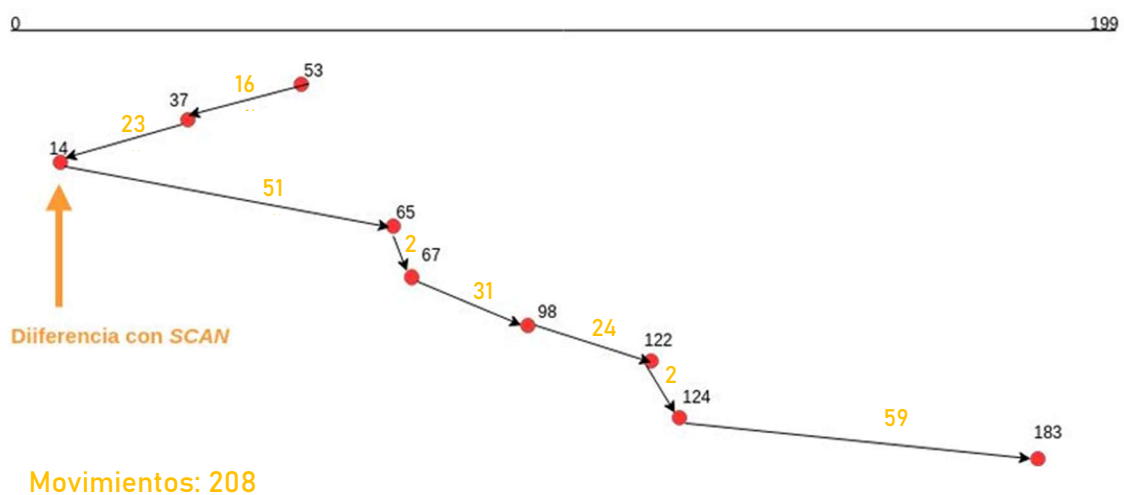
SCAN

Barre el disco en una dirección atendiendo los requerimientos pendientes a esa ruta hasta llegar a **la ultima pista del disco y cambia de dirección**. Es importante saber en que pista se esta y de que pista se viene para determinar el sentido del cabezal.



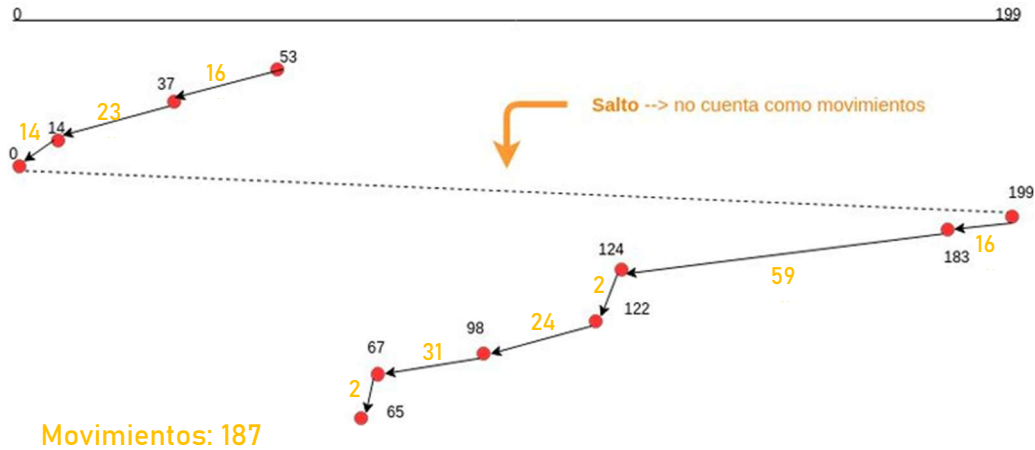
LOOK

Se comporta igual que el SCAN, pero no llega hasta la ultima pista del disco sobre la dirección actual, sino que **llega hasta el ultimo requerimiento de la dirección actual**. Es importante saber en que pista se esta y de que pista se viene para determinar el sentido del cabezal.



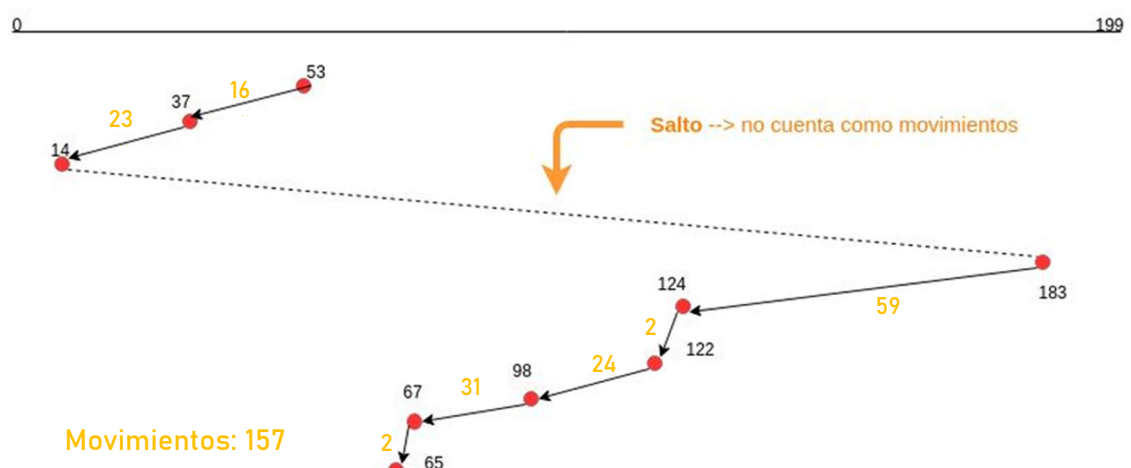
C-SCAN (Circular Scan)

Se comporta igual que el SCAN pero restringe la atención en un solo sentido. **Al llegar a la ultima pista del disco en el sentido actual vuelve a la pista del otro extremo (salto -> no se cuentan los movimientos)** y sigue barriendo en el mismo sentido.



C-LOOK (Circular Look)

Se comporta igual que el LOOK pero restringe la atención en un solo sentido. **Al llegar la ultima pista de los requerimientos en el sentido actual vuelve a la primer pista mas lejana del otro extremo (salto -> no se cuentan los movimientos)** y sigue barriendo en el mismo sentido.



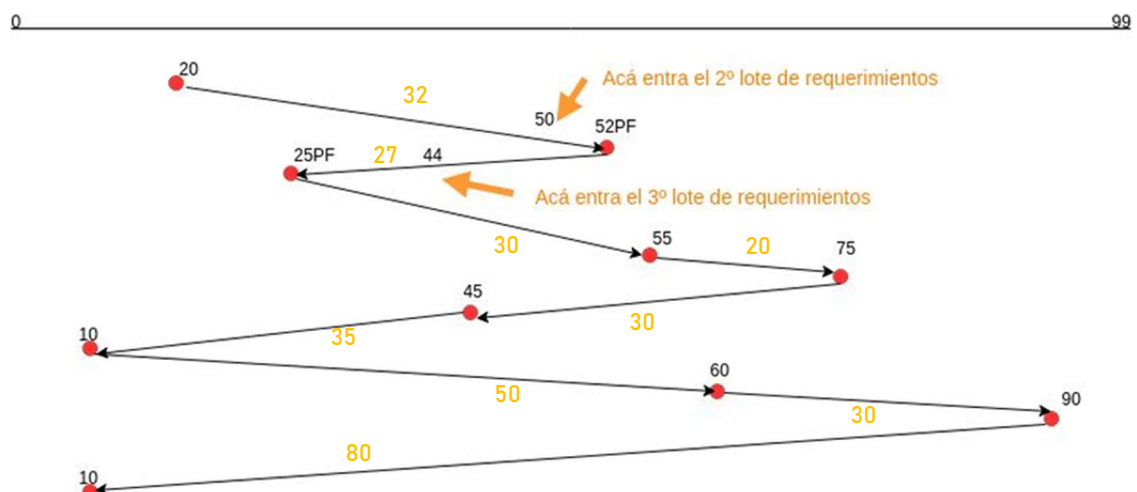
Algoritmos de planificación: Ejemplo con page faults:

Existen requerimientos especiales que deben atenderse con urgencia. Los fallos de página indican simplemente que tienen mayor prioridad con respecto a los requerimientos convencionales, por lo tanto deben ser atendidos inmediatamente después del requerimiento que se está atendiendo actualmente.

FCFS – First Come First Served

Una vez que no existan más requerimientos por page faults en la cola se procede en orden FCFS.

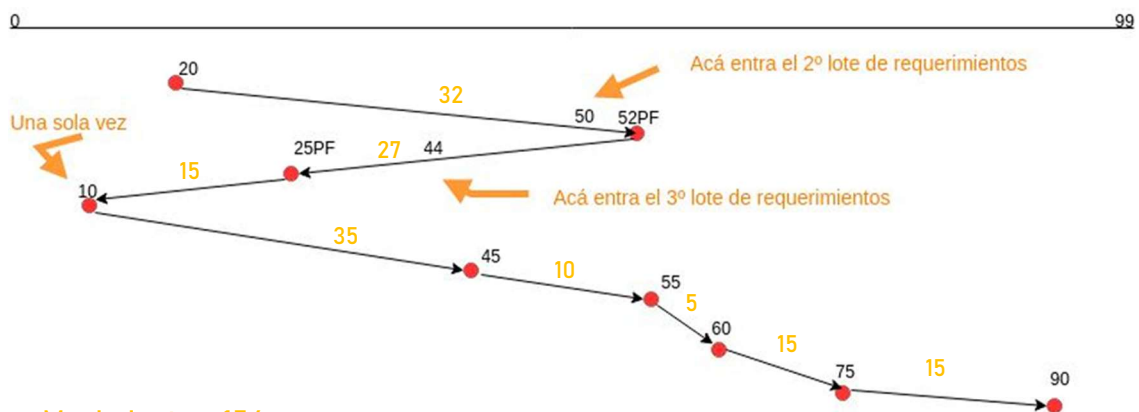
- Cantidad de pistas: 100 (0..99)
- Requerimientos en la cola: {55, 75, 52PF, 45, 10}. Luego de 30 movimientos {25PF, 60} y luego de 10 movimientos más (40 desde el comienzo de la planificación) entra {90, 10}
- Se viene de la pista 15
- Se está atendiendo la pista 20 → izquierda-derecha



Movimientos: 334

SSTF (Shortest Seek Time First)

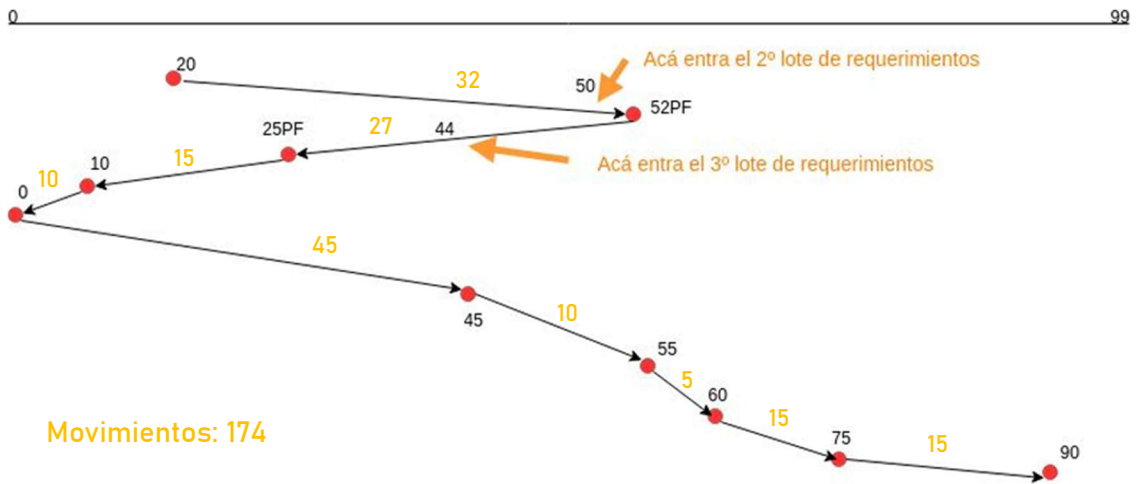
Una vez que no existan más requerimientos por page faults en la cola se procede en orden SSTF.



Movimientos: 154

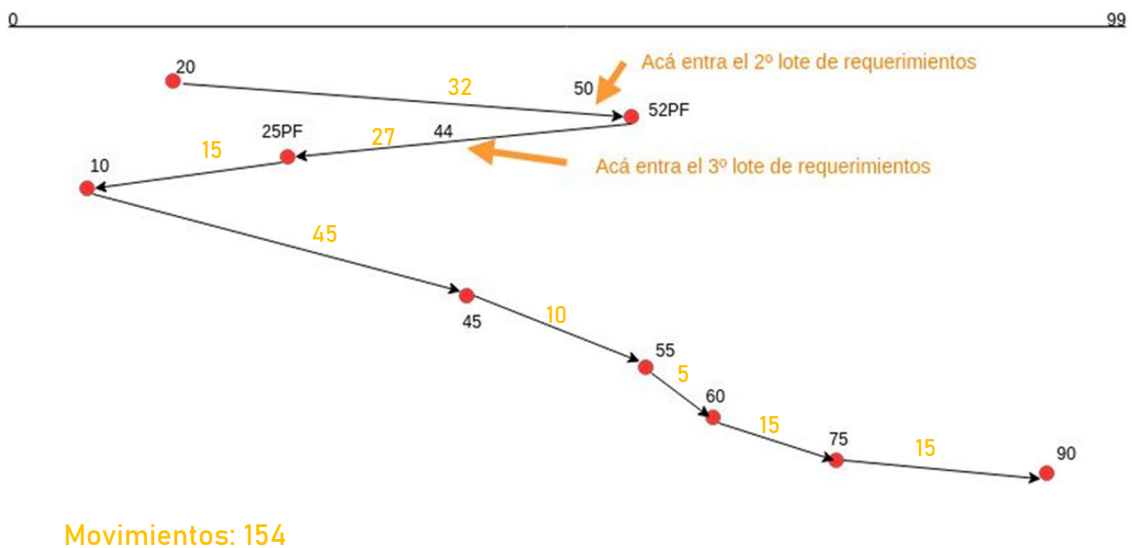
SCAN

Una vez que no existan más requerimientos por page faults en la cola, se procede con el sentido que determina la atención de los últimos dos requerimientos -> **puede cambiar de sentido.**



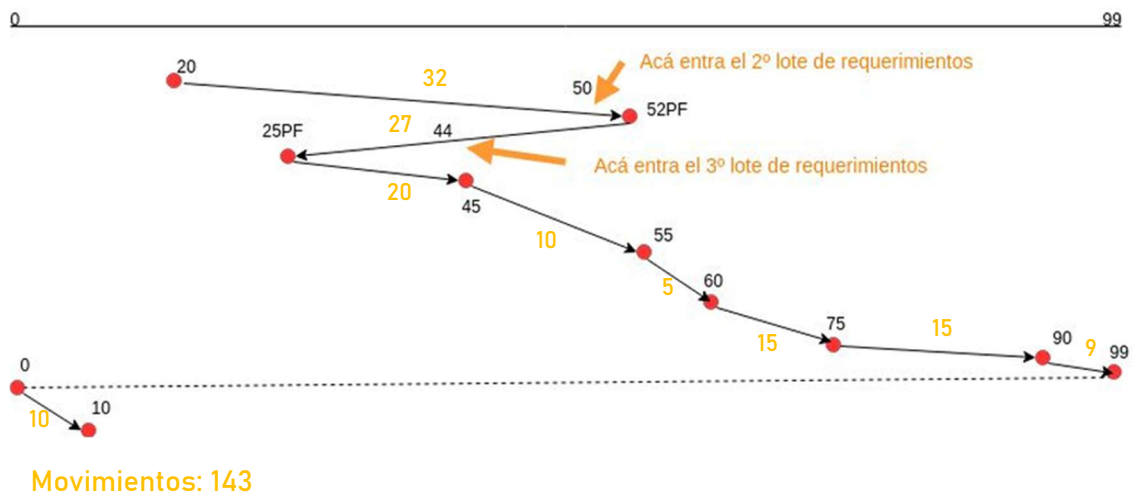
LOOK

Una vez que no existan más requerimientos por page faults en la cola, se procede de la misma manera que en SCAN: con el sentido que determina la atención de los últimos dos requerimientos -> **puede cambiar de sentido.**



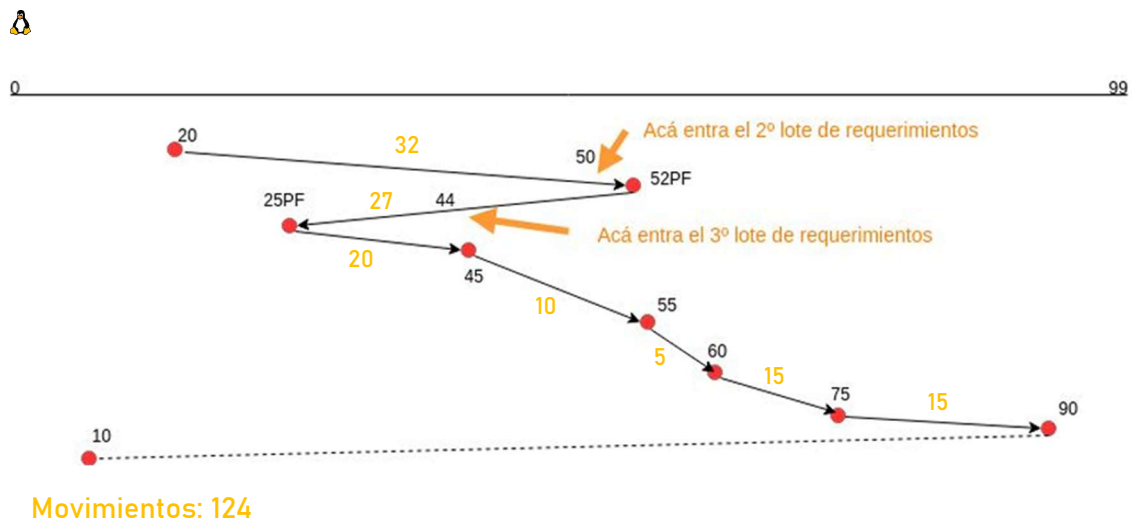
C-SCAN (Circular Scan)

Una vez que no existan más requerimientos por page faults en la cola, se procede con el sentido original -> el sentido no cambia.



C-LOOK (Circular look)

Una vez que no existan más requerimientos por page faults en la cola, se procede del mismo modo que el C-SCAN: con el sentido original -> el sentido no cambia.



🐧 HDD (Hard Disk Drive) (Unidad de disco duro)

Un HDD o disco duro es un dispositivo de almacenamiento que guarda datos de forma permanente como:

- El sistema operativo (Windows, Linux, etc).
- Programas y juegos.
- Fotos, videos y documentos.
- Todo tipo de archivos personales.

Un HDD almacena la información en platos magnéticos que giran a gran velocidad (por ejemplo, 5400 o 7200 RPM) un brazo mecánico con un cabezal lee y escribe los datos sobre esos platos.

Capacidad de un HDD

- **Tamaño de un HDD:** $\#caras * \#pistas_cara * \#sectores_pista * tamaño_sector$

Ejemplo: Supongamos un disco con 6 platos, 2 caras utiles, 1500 pistas por cara y 700 sectores por pista de 256 bytes cada uno:

Tamaño del disco = $(6*2) * 1500 * 700 * 256 = 3.225.600.000$

El resultado lo divido por 2^{30} (para pasarlo a gibibytes)

$3.225.600.000 / 2^{30} = 3.00407$ GiB (Gibibytes)

Tiempo de acceso a un HDD

El tiempo de acceso esta dado por:

- **Seek time** (posicionamiento): Tiempo que tarda en posicionarse la cabeza del cilindro
- **Latency time** (latencia): Tiempo que sucede desde que la cabeza se posiciona en el cilindro hasta que el sector en cuestión pasa por debajo de la misma.
Si este tiempo no se conoce, se considera que es igual a lo que tarda el disco en dar media vuelta. Ej: Disco de 5400 RPM:
 $5400 \text{ vueltas} \rightarrow 1' (1 \text{ minuto}) = 60'' (60 \text{ segundos}) = 60000 \text{ ms} (60 \text{ mil milisegundos})$
 $1 \text{ vuelta} = 60000 \text{ ms} / 5400 = 11,11 \text{ ms} \rightarrow \frac{1}{2} \text{ vuelta} = 5,5 \text{ ms}$
- **Transfer time** (transferencia): Tiempo de transferencia del sector (bloque) del disco a la memoria.

- **Almacenamiento secuencial:**
 $seek + latency + (tiempo_transferencia_bloque * \#bloques)$
- **Almacenamiento aleatorio:**
 $(seek + latency + tiempo_transferencia_bloque) * \#bloques$

Ejemplo: Supongamos un disco con 6 platos, 2 caras utiles, 1500 pistas por cara y 700 sectores por pista de 256 bytes cada uno. El disco gira a 12600 RPM, tiene un tiempo de posicionamiento (seek) de 2 milisegundos y una velocidad de transferencia de 15 MiB/s (Mebibits por segundo)

Se quiere saber cuántos milisegundos se tardarían en transferir un archivo almacenado de manera contigua y aleatoria de 4500 sectores.

- Latencia: $12600 \text{ RPM} \rightarrow 60.000 \text{ ms} / 12.600 = 4.76 \text{ ms} / 2 = 2,38 \text{ ms}$
- Tiempo de transferencia x bloque: $15 \text{ MiB} \rightarrow 1 \text{ s} = 1000 \text{ ms}$
 $15 \text{ MiB} \rightarrow \text{paso a bites } 1 \text{ Mib} = 2^{20} \rightarrow 15 * 2^{20} = 15.728.640 \text{ bits x seg}$
 $256 \text{ bytes tamaño del bloque} \rightarrow \text{paso a bits } 256 * 8 = 2048 \text{ bits}$
 $2048 \text{ bits} * 1000 \text{ ms} = 2.048.000 \text{ bits x ms}$
 $2.048.000 \text{ bits x ms} / 15.728.640 \text{ bits} = 0,1302 \text{ ms}$

Recopilo los datos obtenidos

- Seek time: 2 ms
- Latency time: 2,38 ms
- Tiempo de transferencia x bloque = 0.1302 ms
- #Bloques: 4500

▪ Almacenamiento secuencial: $\text{seek} + \text{latency} + \text{tiempo_transferencia_bloque} * \text{\#bloques} = 2 + 2,3809 + 0,1302 * 4500 = 590,2809 \text{ ms}$

▪ Almacenamiento aleatorio: $(\text{seek} + \text{latency} + \text{tiempo_transferencia_bloque}) * \text{\#bloques} = (2 + 2,3809 + 0,1302) * 4500 = 20299,95 \text{ ms}$

Ocupación sobre un HDD

- **Ocupación de un HDD:** Tamaño del archivo / capacidad de una cara
 $\text{Capacidad de una cara} = \text{\#pistas_cara} * \text{\#sectores_pista} * \text{tamaño_sector}$

Ejemplo: Supongamos un disco con 6 platos, 2 caras utiles, 1500 pistas por cara y 700 sectores por pista de 256 bytes cada uno.

Si queremos calcular cuantas caras ocupara un archivo de 513 Mibytes almacenado de manera contigua a partir del primer sector de la primera pista de una cara determinada:

Calculo la capacidad de una cara = $1500 * 700 * 256 \text{ bytes} = 268.800.000 \text{ bytes}$

Divido el tamaño del archivo por la capacidad de una cara=

$513 \text{ MiB} \rightarrow \text{paso a bytes} = 513 * 2^{20} = 537.919.488$

$537.919.488 / 268.800.000 = 2,00118 \rightarrow \text{redondeo para arriba (no existe un pedazo cara)}$

Filesystem

- **Tamaño del cluster** = tamaño del sector x cantidad de sectores que ocupa el cluster
- **Cantidad de clusters de un archivo** = tamaño del archivo / tamaño del cluster
-> redondeo siempre para arriba.
- **Fragmentación interna de un archivo** = espacio asignado – espacio usado
espacio asignado = cantidad de clusters de un archivo * tamaño del cluster

Ejemplo:

Si se tiene un disco con sectores de tamaño de 512 bytes y un FileSystem que utiliza como unidad de asignación el cluster de 4 sectores. Suponga además que se tienen los archivos: Archivo1 que tiene un tamaño lógico de 28686 bytes y Archivo2 con un tamaño lógico 204855 bytes.

- a. ¿Cuántos clusters necesitan, por separado, Archivo1 y Archivo2 para ser almacenados en el disco?
- b. ¿Cuánto habría de fragmentación interna, por separado, al almacenar Archivo1 y Archivo2? (indicarla en bytes)

Datos

- Sector = 512 bytes
- 1 Cluster = 4 sectores
- Tamaño cluster = $512 * 4 = 2048$ bytes
- Archivo 1 = 28686 bytes - Archivo 2 = 204855 bytes

a. Cantidad de clusters = tamaño del archivo / tamaño cluster

= $28686 / 2048 = 14,0068$ -> redondeo para arriba = 15 clusters para archivo 1

= $204855 / 2048 = 100,0268$ - > redondeo para arriba = 101 clusters para archivo 2

b. Fragmentación interna = espacio asignado – espacio usado

espacio asignado = cant_clusters * tamaño cluster

= $(15 * 2048) = 30720 - 28686 = 2034$ bytes de fragmentación interna para archivo 1

= $(101 * 2048) = 206848 - 204855 = 1993$ bytes de fragmentación interna para archivo 2

👤 Inodos

- **Cantidad de Inodos por bloque** = Tamaño del bloque / Tamaño del inodo
- **Cantidad de direcciones por bloque (o cluster)** = Tamaño del bloque (o cluster) / Tamaño de una dirección
- **Cantidad de bloques de datos** = Tamaño del archivo / Tamaño del bloque -> (redondear para arriba)
- **Tamaño máximo** = DD + DIS + DID(+DIT)

DD N° de directos x tamaño del bloque

DIS N° de indirectos simples x direcciones por bloque x tamaño del bloque

DID N° de indirectos dobles x (direcciones por bloque)² x tamaño del bloque

DIT N° de indirectos triples x (direcciones por bloque)³ x tamaño del bloque

Directo (DD)

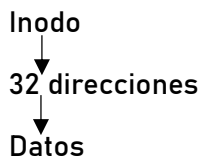
Va directo al dato.



Entonces:

DD = cantidad de punteros x tamaño del bloque

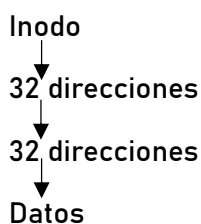
Indirecto simple (DIS)



Hay 1 nivel de direcciones, entonces:

DIS = cantidad de punteros x 32 x tamaño del bloque

Indirecto doble (DID)



Hay 2 niveles, entonces:

DID = cantidad de punteros x 32 x 32 x tamaño del bloque

DD → llega al dato → no multiplico por nada.

DIS → pasa por 1 bloque de direcciones → multiplico una vez.

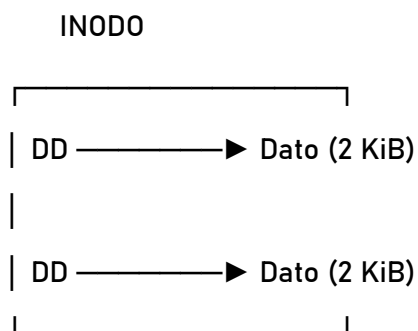
DID → pasa por 2 bloques de direcciones → multiplico dos veces.

DIT → pasa por 3 bloques de direcciones → multiplico tres veces.

Ejemplo:

Inodo con 2 punteros directos (DD), 1 puntero simple (DIS), 1 puntero indirecto doble(DID), donde cada bloque del disco mide 2 KiB y cada bloque de direcciones puede guardar 32 direcciones

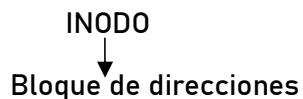
- Los directos (DD) apuntan directamente al dato



Entonces hay 2 bloques de datos y cada uno mide 2 KiB -> $2 \times 2 \text{ KiB} = 4 \text{ Kib}$

- El indirecto simple (DIS)

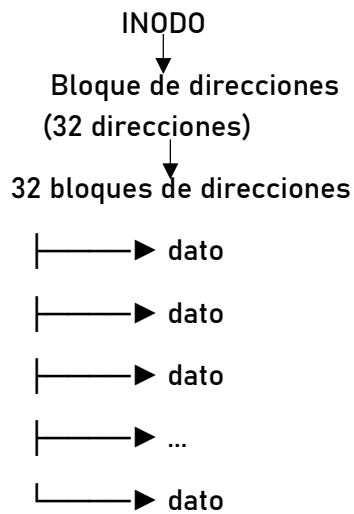
Ahora el puntero no apunta al dato apunta a un bloque lleno de direcciones



Como el bloque tiene 32 direcciones puede apuntar a 32 bloques de datos y cada uno mide 2 KiB -> $32 \times 2 \text{ KiB} = 64 \text{ KiB}$

- El indirecto doble (DID)

Acá aparece un nivel mas en donde el inodo apunta a un bloque y ese bloque apunta a otros bloques y recién esos apuntan a los datos.



El primer bloque tiene 32 caminos, cada uno de esos caminos llega a otro bloque que tiene 32 caminos o sea $\rightarrow (32 \times 32) \times 2 \text{ KiB} = \mathbf{2048 \text{ KiB}}$

Tamaño máximo = 4 + 64 + 2048 = 2116 Kib

Ejemplo:

Se tiene un FileSystem con un modelo de indexado con niveles de indirección, donde se mantiene un i-nodo con 10 índices de direccionamiento directo, 1 índice para indirecto simple y 2 para indirecto doble. Considerando además que utilizan 32 bits para referenciar direcciones de clústeres y que la unidad de asignación es un clúster de tamaño 2 KibiBytes.

- ¿Cuántas direcciones a clusters puede contener un cluster de disco?
- ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?
- ¿Cuántos clusters se necesitan para almacenar un archivo de 55 KibiBytes?

a. Direcciones por cluster = Tamaño del cluster / Tamaño de una dirección.
 $= 2 \times 2^{10} / 4 = 512$

Tamaño del cluster = 2 KiB $\rightarrow 1 \text{ KiB } 2^{10} \text{ bytes} = 2 \text{ KiB} = 2 \times 2^{10} \text{ bytes}$

Tamaño de dirección 32 bits $\rightarrow 8 \text{ bits} = 1 \text{ byte} = 32 / 8 = 4 \text{ bytes}$

Rta: En un cluster entran 512 direcciones

- b. Tamaño máximo del archivo = $DD + DIS + DID = 1.049.620 \text{ KiB}$
 $= 20 + 1024 + 1.048.576$

$DD = 10$ directos $\rightarrow$ cada uno apunta a 1 cluster de 2 KiB
 $= 10 \times 2 \text{ KiB} = 20 \text{ KiB}$

$DIS = 1$ $\rightarrow$ apunta a un cluster con 512 direcciones (calculadas en el anterior)
 $=$ cada dirección apunta a un cluster de 2 KiB $= 512 \times 2 \text{ KiB} = 1024 \text{ KiB}$

$DID = 2$ $\rightarrow$ cada uno apunta a 512 direcciones de las cuales cada una apunta a 512 direcciones que apuntan a clusters de 2 KiB $= 2 \times 512 \times 512 \times 2 \text{ KiB} = 1048576 \text{ KiB}$

Rta: El tamaño máximo que el archivo podría alcanzar es = 1049620 KiB

- c. Archivo de 55 KiB y cada cluster es de 2 KiB, entonces:
 $55 / 2 = 27,5 \rightarrow$ redondeo para arriba 28

Rta: Se necesitan 28 clusters para almacenar al archivo

Prefijos – Equivalencias

Nos permiten representar números largos de manera reducida, Los prefijos binarios nos permiten crear múltiplos binarios (**basados en potencias de 2**).

En la practica de ISO se usa el sistema de prefijos binarios (potencias de 2).

Unidades básicas de información (en bytes)

1 KB = 10^3 bytes

1 MB = 10^6 bytes

1 GB = 10^9 bytes

1 TB = 10^{12} bytes

1 KiB = 2^{10} bytes

1 MiB = 2^{20} bytes

1 GiB = 2^{30} bytes

1 TiB = 2^{40} bytes

1 Kib = 2^{10} bits

1 Mib = 2^{20} bits

1 Gib = 2^{30} bits

1 Tib = 2^{40} bits

1 byte = 8 bits